TypeScript Tutorial #14

Any and Unknown Data Types

1. What is the `any` Data Type?

The `any` data type in TypeScript allows a variable to store **any type of value**.
When a variable is declared with `any`, **TypeScript disables type checking** for that variable.

This means the variable can hold:

- string
- number
- boolean
- object
- array
- anything else

Example

```
let data: any = "Hello";

data = 20;
data = true;
data = { name: "Rahul" };
```

Here, the variable `data` can store different types of values.

2. What Happens When We Use `any`?

When `any` is used, **TypeScript behaves like JavaScript** and stops checking types.

Example

```
let value: any = "Hello";

value.toUpperCase();
value.toFixed();
value.randomFunction();
```

TypeScript will **not show errors**, even if the method does not exist.

However, **runtime errors may occur**.

3. When Should We Use `any` ?

The `any` type should be used **rarely** because it removes TypeScript's main advantage: **type safety**.

Situations where `any` might be used

1. When the type of data is unknown
2. When using third-party libraries
3. When converting a JavaScript project to TypeScript
4. When working with dynamic API data

Example

```
let apiData: any;

apiData = fetchDataFromAPI();
```

4. Problems with Using `any`

Using `any` can cause several issues:

- No type safety

- Bugs are harder to detect
- More runtime errors

Example

```
let price: any = "100";

price.toFixed(2); // Runtime error
```

Here `price` is a string, but `toFixed()` works only with numbers.

5. What is the `unknown` Data Type?

The **`unknown`** type is similar to `any`, but it is **safer**.

It can store any type of value, but **TypeScript requires type checking before using it**.

Example

```
let value: unknown;

value = "Hello";
value = 10;
value = true;
```

6. Type Checking with `unknown`

Before using a variable of type `unknown`, we must check its type.

Example

```
let data: unknown = "Hello";

if (typeof data === "string") {
  console.log(data.toUpperCase());
}
```

Here we check if `data` is a string before calling `toUpperCase()`.

7. Difference Between `any` and `unknown`

Feature	any	unknown
Type Safety	❌ No	✅ Yes
Direct method usage	✅ Allowed	❌ Not allowed
Runtime errors	More likely	Less likely
Recommended	❌ Avoid	✅ Preferred

Example

```
let a: any = "hello";
a.toUpperCase(); // Allowed

let b: unknown = "hello";
b.toUpperCase(); // Error
```

8. Using `unknown` with Type Narrowing

Type narrowing ensures safe usage of unknown values.

Example

```
let input: unknown = 50;

if (typeof input === "number") {
  console.log(input + 10);
}
```

9. Type Assertion with `unknown`

If the developer knows the type, **type assertion** can be used.

Example

```
let data: unknown = "TypeScript";

let str = data as string;

console.log(str.toUpperCase());
```

10. Real Example

```
function processData(value: unknown) {

  if (typeof value === "string") {
    console.log("String:", value.toUpperCase());
  }

  else if (typeof value === "number") {
    console.log("Number:", value + 10);
  }

}
```

Interview Questions

1. What is the `any` data type in TypeScript?

`any` is a data type that allows a variable to hold any type of value and disables type checking.

2. Why should we avoid using `any` ?

Because it removes type safety and increases the chance of runtime errors.

3. What is the `unknown` data type?

`unknown` is a safer version of `any` . It allows any value but requires type checking before usage.

4. What is the difference between `any` and `unknown` ?

`any` allows unrestricted usage without type checking, while `unknown` requires type validation before usage.

5. When should `unknown` be used?

When the type of data is not known but you still want to maintain type safety.

Summary

- `any` disables TypeScript type checking.
- `unknown` is a safer alternative to `any` .

- Avoid using `any` whenever possible.
- Prefer `unknown` when the type is not known.
- Always perform **type checking before using `unknown` values**.